

Initialisierung

Tim Schmitz
05.07.2023

Inhalt

Installation	3
Projekt-Initialisierung	3
Verzeichnisstruktur	4
angular.json	7
ng build	8
ng serve	8
ng lint	9
Checkliste	9

Initialisierung

Die einfachste Möglichkeit der Entwicklung von Web Client-Anwendungen ist die Verwendung der Angular CLI (Command Line Interface). Mit ihr werden alle Werkzeuge installiert und konfiguriert. Die Bedienung erfolgt über die Kommandozeile, in einer Konsole oder in einem Terminal des Texteditors.

Installation

Die Entwicklungsumgebung wird mit folgendem Befehl auf der Kommandozeile installiert:

```
npm install -g @angular/cli
```

Damit wird eine globale Installation der Angular CLI bewirkt. Der folgende Befehl überprüft die Version der Angular CLI.

```
ng --version
```

Mit der Entwicklungsumgebung werden einige Features wie ein automatisierter Build-Prozess mitgeliefert. Weitere Features sind:

- Integration mit TSLint und codelyzer
- Integration mit Testwerkzeugen (Karma für Unit-Tests, Protractor für End-to-End-Tests)
- Integration eines eingebauten Web Servers mit Live-Reload
- Integration mit dem TypeScript-Compiler
- Integration mit dem Angular-Compiler (Übersetzung von Komponenten und Template in JavaScript)

Mit dem folgenden Kommando werden Bausteine generiert:

```
ng generate
```

Das anschließende Kommando gibt an, welche Bausteine in der Angular CLI generiert werden können.

```
ng generate --help
```

Projekt-Initialisierung

Die Initialisierung des Projekts erfolgt im Prinzip mit dem `ng new`-Kommando. Anschließend müssen interaktiv einige Fragen beantwortet werden. Danach legt die Entwicklungsumgebung die nötigen Konfigurationsdateien an und installiert mit dem Node Package Manager die Abhängigkeiten. Das neue Projekt befindet sich in einem Unterverzeichnis des aktuellen Verzeichnisses. Dabei wird der Projektname als Verzeichnisname verwendet. Abgesehen von den Konfigurationsdateien wird ein Gerüst oder Skelett von Verzeichnissen sowie rudimentärer Beispielcode angelegt. Mit dem folgenden Kommando wird die Beispielanwendung "contacts-manager" direkt angelegt, d.h. ohne den interaktiven Modus, aber mit Benennung des Projektnamens.

```
ng new --interactive=false contacts-manager
```

Mit dem folgenden Befehl werden Endungen für Stylesheet-Dateien definiert.

```
ng new --style=scss
```

Standardmäßig wird für Angular-Projekte das Format CSS (Cascading Stylesheets) mit der Endung `.css` verwendet. Weitere Optionen werden mit dem Befehl

```
ng new --help
```

angezeigt. Wenn man bereits ein existierendes Angular-Projekt besitzt, dann werden mit

```
npm install
```

alle Abhängigkeiten des Projekts installiert.

Verzeichnisstruktur

Die Verzeichnisstruktur, die von Angular bei der Initialisierung des Projekts angelegt wurde, hat folgendes Aussehen:

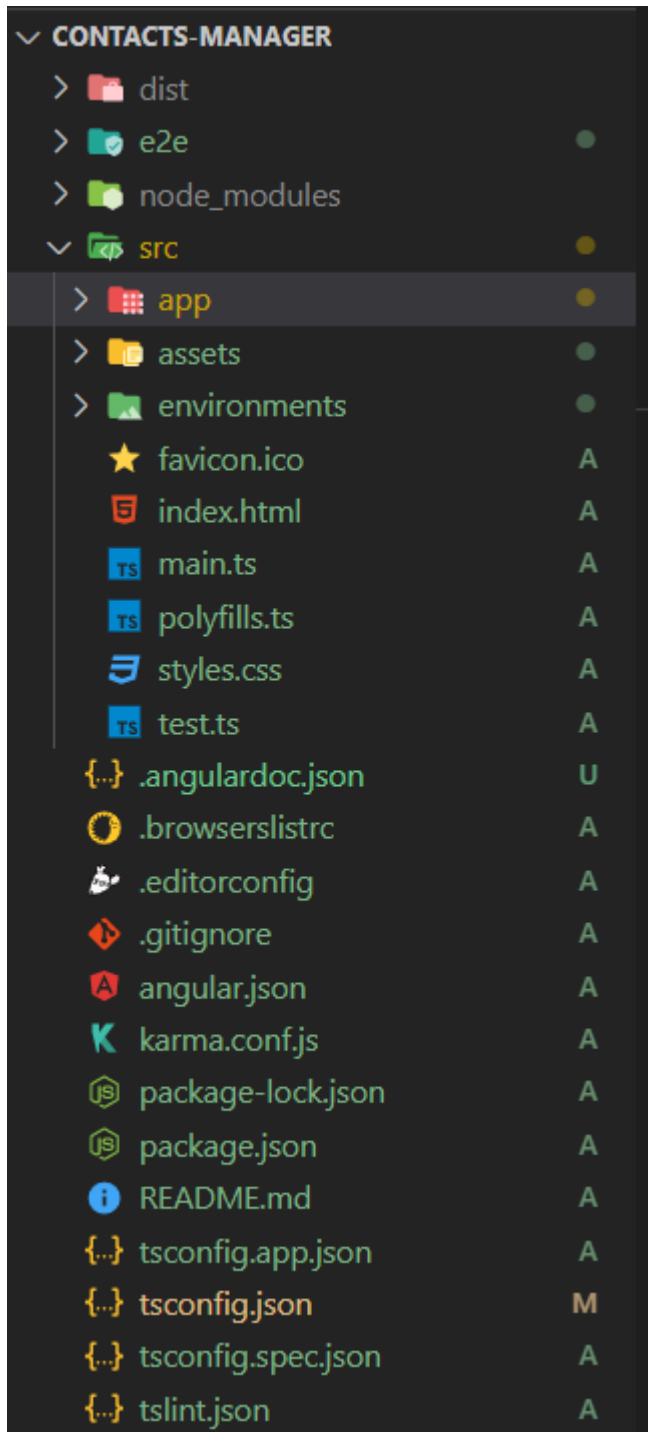


Abbildung 1: Verzeichnisstruktur des Projekts

Auf der obersten Ebene existieren vier Verzeichnisse und viele Dateien im Hauptverzeichnis.

dist

Dieses Verzeichnis beinhaltet den Code, der bei einem Build erzeugt wird.

node_modules

Hier kann man auch lokale npm-Pakete definieren, die nur innerhalb des Projekts in dem Verzeichnis `node_modules` verwendet werden.

e2e

Dieses Verzeichnis beinhaltet Code und Konfigurationsdateien für die End-to-End-Tests.

src

Dieser Ordner enthält alle Dateien, die für die Anwendung benötigt werden wie zum Beispiel Bilder oder Schriftarten sowie den gesamten Quellcode der Anwendung.

Folgende Dateien befinden sich im Hauptverzeichnis:

.angulardoc.json

Diese Datei wurde generiert von der Erweiterung AngularDoc der Entwicklungsumgebung Visual Studio Code. Sie ist nur interessant, wenn ein Repository benutzt wird und wenn mehrere Entwickler den Service von AngularDoc verwenden. Für Alleinstellungs-Projekte hat die Datei keine Bedeutung.

.editorconfig

Diese Datei ist eine Konfigurationsdatei für Texteditoren. Dort werden Einstellungen vorgenommen, die nicht Angular-spezifisch sind, wie der verwendete Zeichensatz oder die Anzahl der Stellen, die bei Einrückungen definiert werden.

.browserlistrc

Diese Datei wird vom Build verwendet, um die CSS- und JavaScript-Ausgaben auf die verschiedenen Browser (Chrome, Firefox, Edge, Safari) einzustellen. Der Internet Explorer wird nicht unterstützt.

.gitignore

Als Standard erzeugt die Angular-Entwicklungsumgebung ein git-Repository für jedes Projekt. Mit der folgenden Option kann die Erstellung eines Repositories verhindert werden:

```
ng new --skip-git
```

In jedem Fall wird ein `.gitignore`-Datei angelegt. Diese beinhaltet alle (generierten) Dateien, die nicht in das Repository hochgepuscht werden.

angular.json

Diese Datei ist die Konfigurationsdatei der Angular CLI. Sie enthält - wie man später sieht - Optionen zur Steuerung der Entwicklungsumgebung.

karma.conf.js

Diese Datei ist die Konfigurationsdatei von Karma, dem Testwerkzeug der Angular-Entwicklungsumgebung für Unit-Tests.

package.json

Diese Datei ist die Konfigurationsdatei von npm mit Abhängigkeiten und Versionen.

tsconfig*.json

Es gibt insgesamt drei Konfigurationsdateien für den Typescript-Compiler. Die Datei tsconfig.json ist die zentrale Konfigurationsdatei für den TypeScript-Compiler. Die Datei tsconfig.app.json ist eine anwendungsspezifische Datei, falls in dem Projekt mehrere Applikationen entwickelt werden. Die Datei tsconfig.spec.json ist eine Datei, die für die Spezifikation von Tests verwendet wird.

tslint.json

Diese Datei ist die globale Konfigurationsdatei von TSLint für das Angular-Projekt.

In dem End-to-End-Ordner e2e existieren die folgenden Dateien und Verzeichnisse:

src

In diesem Verzeichnis wird der Code für die End-to-End-Tests in einem Angular-Projekt gehalten.

protractor.conf.js

Diese Datei ist die Konfigurationsdatei von Protractor, dem Testwerkzeug für Angular-Projekte für End-to-End-Tests.

tsconfig.e2e.json

Diese Datei ist die Konfigurationsdatei für den Typescript-Compiler im Fall von End-to-End-Tests. Sie wird ergänzend zur Datei tsconfig.json verwendet.

Das Verzeichnis src beinhaltet im weitesten Sinne den Source Code der Anwendung.

app

In diesem Verzeichnis wird der Code der Anwendung verwaltet. Hier werden die Entwickler ihre Arbeitsartefakte editieren.

assets

Dieses Verzeichnis beinhaltet in der Regel statische Ressourcen wie Bilder oder Fonts.

environments

Dieses Verzeichnis enthält Konfigurationsdateien für den Build der Anwendung, die umgebungsspezifisch sind (Produktion oder Test).

favicon.ico

Das Favicon-Icon wird standardmäßig von manchen Browsern versucht zu laden - egal, ob vorhanden oder nicht. Daher ist es möglicherweise eine gute Idee, eine Art Dummy-Objekt hier zu hinterlegen.

index.html

Die Datei index.html ist die Hauptdatei einer jeden Single-Page-Application (SPA). Wird auf die entsprechende URL geroutet, so lädt der Browser genau dieses Datei.

main.ts

Die Datei main.ts ist die Haupt-TypeScript-Datei für die Anwendung. Genau wie in Java-Projekten existiert hier genau eine Hauptklasse.

polyfills.ts

Die Datei für Polyfills ist ein Fallback für ältere Browser, falls man neuere Features damit nutzen will. Es ist hier auch möglich, eigene Polyfills zu definieren.

styles.css

Die Datei styles.css erlaubt es, globale Stylesheets zu definieren. Standardmäßig ist diese Datei allerdings leer.

test.ts

Diese Datei ist die Hauptdatei für Unit-Tests.

angular.json

Die Konfigurationsdatei angular.json wird bei der Initialisierung eines Projekts angelegt. Sie beinhaltet Optionen, die das Verhalten der Entwicklungsumgebung steuern. Dabei ist die Datei in verschiedene Bereiche aufgespalten.

```
{
  ...,
  "version": 1,
  "newProjectRoot": "projects",
  "projects": {
    "contacts-manager": {
      "projectType": "application",
      "schematics": {},
      "root": "",
      "sourceRoot": "src",
      "prefix": "app",
      "architect": {...}
    },
  },
}
```

Die version-Eigenschaft gibt das Format der Versionierung an. Die projects-Eigenschaft beschreibt den Bereich der Projektkonfigurationen. Seine Eigenschaften sind die verschiedenen Projekte, die zum Workspace der Entwicklungsumgebung gehören. In diesem Fall existiert lediglich das Projekt "contacts-manager".

Ein Projekt selber wird in verschiedene Bereiche aufgeteilt. Die schematics-Eigenschaft legt die Verwendung des ng generate- Kommandos fest. Generell können die meisten Optionen, die man hier festlegen kann, auch als Parameter über die Kommandozeile mitgegeben werden.

Die architect-Eigenschaft ist wichtig für die Konfiguration der Angular-Kommandos ng build und ng serve. Für jedes dieser Kommandos gibt es jeweils eine builder-, eine options- und eine configurations-Eigenschaft.

```
"architect": {
  "build": {
    "builder": "@angular-devkit/build-angular:browser",
    "options": {...}
    "configurations": {
      "production": {...}
    }
  },
  ...,
}
```

Die builder-Eigenschaft gibt den eigentlichen Namen des Kommandos an, wenn ng build aufgerufen wird. Er besteht aus dem Paketnamen und dem Builder-Namen.

Im options-Objekt befinden sich Optionen für das Kommando. Im configurations-Objekt werden Standardoptionen überschrieben oder ergänzt - wie zum Beispiel den Bau einer Produktionsversion.

ng build

Das Kommando ng build führt zum Bau der Anwendung und schreibt das Ergebnis in den Ordner dist in ein Verzeichnis, das genauso heißt wie die Anwendung. In der Regel wird ng build verwendet, wenn es schon einen Webserver gibt, der die Anwendung ausliefert. Man muss allerdings dann darauf achten, dass der Webserver in dem Verzeichnis gestartet wird, in dem die Single Page Application läuft. Ansonsten erhält man ständig die HTTP-Response 404 (Not found) zurück, da der Server versucht, Seiten zu laden, die in einem anderen Verzeichnis liegen als in demjenigen, in dem der Server läuft.

```
cd C:\Users\schmi\Daten\Web\contacts-manager\dist\contacts-manager
http-server
```

Der komplette TypeScript-Code wird nach JavaScript übersetzt und in eine einzige Datei geschrieben (hier: main.js). Sie heißt das Code-Bündel, und ist die Datei, die vom Server angefordert wird, wenn er index.html lädt.

Unter anderem verfügt ng build über folgende Optionen:

--configuration=production

Mit dieser Option kann man die Eigenschaften des configurations-Objekt aus der angular.json verwenden, dann wird etwa hier die entsprechende Datei aus dem environment-Verzeichnis übernommen (environment.prod.ts im Gegensatz zu environment.ts sonst).

--aot

Mit dieser Option verwendet ng build den Ahead-of-Time-Compiler, der Komponenten bereits beim Programmieren kompiliert und nicht erst zur Laufzeit im Browser, was die Früherkennung von Fehlern unterstützt und mehr Sicherheit bietet.

--prod

Dieses Kommando wird verwendet, eine Produktionsversion der Anwendung zu bauen, das übrigens identisch mit dem ersten Kommando ist.

--help

Diese Option ist geboten, wenn man alle Optionen aufgelistet haben möchte.

ng serve

Das Kommando ng serve wird aufgerufen, wenn man eine Anwendung baut und direkt danach einen Webserver startet, der die Anwendung ausliefert. Es wird benutzt, wenn die Anwendung entwickelt wird. Der erzeugte Code wird nicht - wie beim ng build - auf die Festplatte geschrieben, sondern "on the fly" erzeugt und im Arbeitsspeicher gehalten. Daher werden alle Änderungen im Code direkt zu einem automatischen Neubau der Anwendung führen. Kompilierfehler werden daher auch sofort in der Konsole angezeigt, falls sie nicht schon sofort im Texteditor angezeigt werden. Die fertige Anwendung wird unter folgender URL im Browser angezeigt:

```
http://localhost:4200
```


Neben allen Optionen des ng build-Kommandos bietet ng serve eigene Optionen:

--port=8080

Mit dieser Option kann man einen anderen Webserver als den eingebauten von Angular verwendet.

--ssl=true

Diese Option erlaubt es, die Anwendung über das sichere Http-Protokoll mit https zu erreichen. Ein Zertifikat wird von der Entwicklungsumgebung bereitgestellt.

--ssl-key

Hiermit wird der SSL-Schlüssel für die ssl-Option spezifiziert.

--ssl-cert

Hiermit wird das SSL-Zertifikat für die ssl-Option spezifiziert.

ng lint

Das Kommando ng lint wird aufgerufen, wenn man eine Anwendung qualitätssichern will. Das Tool TSLint untersucht daraufhin den Applikationscode.

Checkliste

Folgende Checks sollten bei der Initialisierung eines neuen Projekts ausgeführt werden

1. Installation von Node.js und npm (in Node.js enthalten)
2. Installation der Entwicklungsumgebung mit npm install -g @angular/cli
3. Initialisierung des Projekts mit ng new --interactive=false contacts-manager
4. Anpassung von tsconfig.json für Compiler-Optionen
5. Erstes Starten mit ng serve
6. Kleine Code-Änderungen an src/app/app.component.html
7. Testen von ng lint und ng build
8. Spätere Abhängigkeiten mit npm install auflösen